

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

Звіт з виконання лабораторної роботи

Дослідження вимог безпеки OWASP та формування аналогічних вимог для
децентралізованих додатків на базі Ethereum

Виконали студенти групи ФІ-52мн:
Волошин Ігор
Геращенко Володимир
Дорошенко Юрій

Київ 2026

Мета роботи

Отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні. Дослідження вимог OWASP щодо безпеки веб-додатків і складання аналогічних вимог для системи децентралізованих додатків на базі блокчейну Ethereum.

1 Вступ

1.1 Загальна характеристика OWASP

OWASP (Open Web Application Security Project) — це некомерційний міжнародний проєкт, метою якого є покращення безпеки програмного забезпечення. Організація публікує стандарти, методології та документи, зокрема добре відомий список **OWASP Top 10** — перелік десяти найнебезпечніших вразливостей веб-додатків.

Актуальна редакція OWASP Top 10 (2021) визначає такі категорії ризиків:

- A01:** Broken Access Control (Порушення контролю доступу)
- A02:** Cryptographic Failures (Криптографічні збої)
- A03:** Injection (Ін'єкції)
- A04:** Insecure Design (Небезпечне проєктування)
- A05:** Security Misconfiguration (Неправильна конфігурація безпеки)
- A06:** Vulnerable and Outdated Components (Вразливі та застарілі компоненти)
- A07:** Identification and Authentication Failures (Збої ідентифікації та автентифікації)
- A08:** Software and Data Integrity Failures (Збої цілісності ПЗ та даних)
- A09:** Security Logging and Monitoring Failures (Збої журналювання та моніторингу)
- A010:** Server-Side Request Forgery — SSRF (Підrobка запитів на стороні сервера)

1.2 Обрана система децентралізованих додатків

Для виконання роботи обрано **Ethereum** — найбільш розповсюджену платформу для розробки децентралізованих додатків (DApps) і смарт-контрактів. Смарт-контракти Ethereum написані мовою **Solidity** і виконуються в **Ethereum Virtual Machine (EVM)**.

Особливості платформи, що впливають на безпеку:

- **Незмінність коду** — після розгортання контракт неможливо виправити без використання спеціальних патернів (проху, upgradeable).
- **Публічність стану** — всі дані в блокчейні є загальнодоступними.
- **Фінансові ризики** — контракти часто зберігають та переміщують реальні кошти (ETH, ERC-20 токени).
- **Вартість операцій** — кожна транзакція потребує оплати газу (Gas), що відкриває специфічні вектори атак.
- **Конкурентне середовище** — транзакції обробляються публічно, що дозволяє зловмисникам маніпулювати їх порядком (MEV, front-running).

2 Аналіз вимог OWASP Top 10 (2021)

У таблиці 1 наведено перелік вразливостей OWASP Top 10 з їх коротким описом та наслідками для традиційних веб-додатків.

Табл. 1: Вразливості OWASP Top 10 (2021) для веб-додатків

ID	Назва	Опис	Наслідки
A01	Broken Access Control	Неналежна перевірка прав доступу дозволяє користувачам виконувати дії поза межами своїх привілеїв	Несанкціонований доступ, модифікація або видалення даних
A02	Cryptographic Failures	Слабкі або неправильно реалізовані криптографічні механізми захисту даних	Витік конфіденційних даних, злом паролів
A03	Injection	Впровадження шкідливого коду через непідтверджені вхідні дані (SQL, OS, LDAP)	Виконання довільного коду, злив даних
A04	Insecure Design	Відсутність загрозоорієнтованого моделювання на етапі архітектури	Системні вразливості, що неможливо усунути без перепроєктування
A05	Security Misconfiguration	Неправильна конфігурація сервісів, хмарних сховищ, заголовків безпеки тощо	Розкриття чутливої інформації, несанкціонований доступ
A06	Vulnerable and Outdated Components	Використання бібліотек і фреймворків з відомими вразливостями	Компрометація всього стеку застосунку
A07	Identification and Authentication Failures	Вразливості в механізмах входу, управлінні сесіями та багатофакторній автентифікації	Перехоплення акаунтів, брут-форс
A08	Software and Data Integrity Failures	Відсутність перевірки цілісності коду, даних конвеєра CI/CD та оновлень	Атаки на ланцюг постачання, підміна коду
A09	Security Logging and Monitoring Failures	Відсутність або неналежна реалізація журналювання і сповіщення про інциденти	Невиявлені атаки, неможливість криміналістичного аналізу
A10	Server-Side Request Forgery	Сервер виконує запити до довільних URL за вказівкою зловмисника	Сканування внутрішньої мережі, доступ до метаданих хмари

3 Аналогічні вимоги безпеки для DApps на Ethereum

Нижче наведено десять вимог безпеки, аналогічних OWASP Top 10, але адаптованих до специфіки смарт-контрактів і децентралізованих додатків на платформі Ethereum. Для кожної вимоги зазначено відповідний аналог OWASP, опис вразливості та рекомендації щодо її усунення.

3.1 DC01 — Порушення контролю доступу до функцій контракту

Аналог OWASP: A01 Broken Access Control

Опис: У смарт-контрактах функції з модифікаторами `public` або `external` без додаткової перевірки прав можуть бути викликані будь-якою адресою. Це особливо небезпечно для адміністративних функцій (`mint`, `pause`, `withdraw`, `setOwner`).

Приклад вразливого коду:

Лістинг 1: Function without access control check

```
1 // VULNERABLE: anyone can withdraw funds
2 function withdraw(uint256 amount) public {
3     payable(msg.sender).transfer(amount);
4 }
```

Рекомендації:

- Використовувати модифікатор `onlyOwner` або бібліотеку `OpenZeppelin AccessControl` / `Ownable`.
- Впроваджувати рольову модель доступу (Role-Based Access Control, RBAC).
- Для критичних операцій застосовувати мультипідпис (`multisig`).

3.2 DC02 — Некоректна обробка чутливих даних у стані

Аналог OWASP: A02 Cryptographic Failures

Опис: Всі дані в сховищі (`storage`) смарт-контракту публічно доступні в блокчейні, навіть якщо змінна оголошена як `private`. Збереження паролів, ключів або приватної інформації без шифрування є критичною вразливістю.

Рекомендації:

- Ніколи не зберігати секрети on-chain. Натомість зберігати лише хеші (наприклад, `keccak256`).
- Для конфіденційних даних використовувати рішення off-chain із криптографічними доказами (ZK-proofs).
- Перед розгортанням проводити аналіз усіх `storage`-слотів.

3.3 DC03 — Атака повторного входу (Reentrancy)

Аналог OWASP: A03 Injection

Опис: Reentrancy — одна з найвідоміших вразливостей Ethereum (саме вона призвела до злому The DAO у 2016 р., збиток \$60 млн). Під час виклику зовнішнього контракту зловмисник може рекурсивно повторити виклик функції до оновлення стану.

Приклад вразливого коду:

Лістинг 2: Vulnerable withdraw function (reentrancy)

```
1 // VULNERABLE: state updated AFTER external call
2 function withdraw() public {
3     uint256 bal = balances[msg.sender];
4     require(bal > 0);
5     (bool ok,) = msg.sender.call{value: bal}(""); // external call
6     require(ok);
7     balances[msg.sender] = 0; // too late!
8 }
```

Рекомендації:

- Застосовувати патерн Checks-Effects-Interactions: спочатку перевірки, потім зміна стану, потім зовнішній виклик.
- Використовувати модифікатор `nonReentrant` з OpenZeppelin.
- Уникати `call{}` там, де достатньо `transfer()` або `send()`.

3.4 DC04 — Небезпечне проєктування бізнес-логіки

Аналог OWASP: A04 Insecure Design

Опис: Через незмінність смарт-контрактів архітектурні вади є особливо руйнівними. Відсутність механізмів аварійної зупинки (*circuit breaker*), оновлення логіки або повернення помилкових транзакцій призводить до неவிправних втрат.

Рекомендації:

- На етапі проєктування застосовувати threat modeling специфічно для EVM.
- Передбачати механізм `pause()` для аварійної зупинки.
- Розглядати оновлювані контракти (Transparent Proxy, UUPS) для можливості виправлення критичних помилок.
- Проводити формальну верифікацію логіки перед розгортанням.

3.5 DC05 — Неправильна конфігурація розгортання

Аналог OWASP: A05 Security Misconfiguration

Опис: Помилкова конфігурація при розгортанні включає: ненульові привілейовані адреси (`owner`) на тестових ключах, увімкнена отладка в продакшн, некоректні параметри ініціалізації проксі-контракту.

Рекомендації:

- Використовувати скрипти розгортання з явною перевіркою параметрів.
- Передавати права адміністратора на мультисиг-гаманець після розгортання.
- Перевіряти конфігурацію через публічні блокчейн-оглядачі (Etherscan) перед передачею в продакшн.

3.6 DC06 — Використання застарілих або небезпечних бібліотек

Аналог OWASP: A06 Vulnerable and Outdated Components

Опис: Застарілі версії бібліотек OpenZeppelin, Chainlink або інших залежностей можуть містити відомі вразливості. Форки або неофіційні копії бібліотек збільшують ризик компрометації.

Рекомендації:

- Використовувати виключно офіційні, актуальні версії OpenZeppelin та перевіряти їх через `npm audit`.
- Підключати бібліотеки через `npm/Hardhat`, а не копіюванням коду.
- Слідкувати за бюлетенями безпеки (OpenZeppelin Security Advisories).

3.7 DC07 — Збої автентифікації та верифікації підпису

Аналог OWASP: A07 Identification and Authentication Failures

Опис: Смарт-контракти автентифікують учасників через `msg.sender`. Вразливості виникають при некоректній верифікації підписів (`ecrecover`), відсутності захисту від replay-атак або при делегованих викликах (`delegatecall`).

Рекомендації:

- При перевірці підписів завжди включати `chainId` і попси для захисту від replay-атак (стандарт EIP-712).
- Перевіряти, що `ecrecover` не повертає нульову адресу.
- З обережністю використовувати `delegatecall`: це виконує чужий код у контексті поточного контракту.

3.8 DC08 — Маніпуляції з ціновими оракулами та зовнішніми даними

Аналог OWASP: A08 Software and Data Integrity Failures

Опис: Смарт-контракти DeFi часто покладаються на зовнішні джерела цін (оракули). Маніпуляція `price oracle` через флеш-позики або централізований оракул призводить до критичних збитків (атака на Mango Markets, 2022 р., \$117 млн).

Рекомендації:

- Використовувати децентралізовані оракули (Chainlink, TWAP).
- Перевіряти свіжість даних оракула (перевіряти `updatedAt`).
- Уникати спот-цін з DEX як єдиного джерела, застосовувати часово-зважені середні ціни (TWAP).

3.9 DC09 — Недостатня прозорість та відсутність моніторингу

Аналог OWASP: A09 Security Logging and Monitoring Failures

Опис: На відміну від традиційних додатків, всі транзакції в Ethereum публічні, проте відсутність `event`-журналювання всередині контракту ускладнює аудит і виявлення аномалій.

Рекомендації:

- Генерувати **event** для всіх значущих дій: переказів, зміни ролей, призупинення контракту тощо.
- Налаштовувати зовнішній моніторинг on-chain (Tenderly, OpenZeppelin Defender) з оповіщеннями про підозрілі транзакції.
- Регулярно перевіряти емітовані події через Etherscan / The Graph.

3.10 DC10 — Залежність від зовнішніх викликів та front-running

Аналог OWASP: A10 Server-Side Request Forgery

Опис: Транзакції Ethereum публічно видимі в мемпулі до їх включення в блок. Зловмисники можуть: відстежувати вигідні транзакції й розміщувати власні з вищою ціною газу (front-running); маніпулювати порядком транзакцій (sandwich attack) для отримання прибутку за рахунок жертви.

Рекомендації:

- Для чутливих операцій використовувати схему *commit-reveal*: спочатку фіксувати хеш зобов'язання, потім відкривати значення.
- Задавати допустимий відсоток прослизання (slippage tolerance) в транзакціях DEX.
- Розглядати приватні мемпули (Flashbots Protect RPC) для захисту від MEV-атак.

4 Порівняльна таблиця вимог

Табл. 2: Відповідність між OWASP Top 10 та вимогами безпеки Ethereum DApps

OWASP	OWASP (Web)	DApp	Вразливість DApp	Рекомендований захист
A01	Broken Access Control	DC01	Функції без перевірки прав	RBAC, <code>onlyOwner</code> , <code>multisig</code>
A02	Cryptographic Failures	DC02	Секрети у <code>storage</code>	Зберігати лише хеші, ZK-proofs
A03	Injection	DC03	Reentrancy-атака	CEI-патерн, <code>nonReentrant</code>
A04	Insecure Design	DC04	Відсутність <code>circuit breaker</code>	Threat modeling, Proxy-патерн
A05	Security Misconfiguration	DC05	Помилки конфігурації розгортання	Скрипти деплою, перевірка на Etherscan
A06	Vulnerable Components	DC06	Застарілі залежності	Актуальні версії OpenZeppelin
A07	Auth Failures	DC07	Слабка верифікація підпису	EIP-712, <code>nonce</code> , перевірка <code>ecrecover</code>
A08	Integrity Failures	DC08	Маніпуляція з <code>ораклом</code>	Chainlink TWAP, перевірка свіжості
A09	Logging Failures	DC09	Відсутність <code>event-журналювання</code>	Events, OZ Defender, Tenderly
A10	SSRF	DC10	Front-running у мемпулі	Commit-reveal, Flashbots, slippage

5 Практична демонстрація

5.1 Reentrancy (DC03) — вразливий vs захищений контракт

Нижче наведено порівняння вразливої та захищеної реалізацій функції зняття коштів у смарт-контракті Solidity.

Вразлива реалізація:

Лістинг 3: VulnerableBank: external call before state update

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract VulnerableBank {
5     mapping(address => uint256) public balances;
6
7     function deposit() external payable {
8         balances[msg.sender] += msg.value;
9     }
10
11     // VULNERABLE: state updated AFTER external call
12     function withdraw() external {
13         uint256 amount = balances[msg.sender];
14         require(amount > 0, "No balance");
15         (bool success,) = msg.sender.call{value: amount}("");
16         require(success, "Transfer failed");
17         balances[msg.sender] = 0; // too late -- reentrancy already
18                                     possible
19     }
20 }
```

Захищена реалізація:

Лістинг 4: SecureBank: Checks-Effects-Interactions + nonReentrant

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
5
6 contract SecureBank is ReentrancyGuard {
7     mapping(address => uint256) public balances;
8
9     function deposit() external payable {
10         balances[msg.sender] += msg.value;
11     }
12
13     // SECURE: Checks-Effects-Interactions + nonReentrant
14     function withdraw() external nonReentrant {
15         // 1. Checks
16         uint256 amount = balances[msg.sender];
17         require(amount > 0, "No balance");
18         // 2. Effects (state updated BEFORE external call)
19         balances[msg.sender] = 0;
20         // 3. Interactions
```

```

21         (bool success,) = msg.sender.call{value: amount}("");
22         require(success, "Transfer_failed");
23
24         emit Withdrawn(msg.sender, amount);
25     }
26
27     event Withdrawn(address indexed user, uint256 amount);
28 }

```

5.2 Контроль доступу (DC01) — реалізація RBAC

Лістинг 5: RBAC with OpenZeppelin AccessControl

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  import "@openzeppelin/contracts/access/AccessControl.sol";
5
6  contract RBACDemo is AccessControl {
7      bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
8      bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
9
10     bool public paused;
11
12     constructor() {
13         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
14         _grantRole(PAUSER_ROLE, msg.sender);
15     }
16
17     function mint(address to, uint256 amount)
18         external
19         onlyRole(MINTER_ROLE)
20     {
21         require(!paused, "Contract_is_paused");
22         // minting logic...
23     }
24
25     function pause() external onlyRole(PAUSER_ROLE) {
26         paused = true;
27         emit Paused(msg.sender);
28     }
29
30     event Paused(address indexed account);
31 }

```

Висновок

У ході лабораторної роботи було досліджено стандарт безпеки OWASP Top 10 для традиційних веб-додатків та складено аналогічний перелік із десяти вимог безпеки для децентралізованих додатків на базі платформи Ethereum.

Проведений аналіз показав, що між безпекою веб-додатків і смарт-контрактів існує структурна аналогія: кожній категорії ризику OWASP відповідає специфічний вектор атаки в екосистемі EVM. Разом з тим Ethereum-середовище має унікальні характеристики — незмінність коду, публічність стану, фінансові активи on-chain і конкурентне середовище мемпулу — що вимагає окремого, більш суворого підходу до безпеки порівняно з класичними веб-системами.

Ключові висновки:

1. Reentrancy (DC03) залишається найнебезпечнішою вразливістю для фінансових контрактів. Патерн Checks-Effects-Interactions разом із модифікатором `nonReentrant` є обов’язковим стандартом.
2. Контроль доступу (DC01) повинен реалізовуватися через рольову модель (RBAC) з мінімальними привілеями і мультисиг-адміністраторами.
3. Відсутність конфіденційності даних on-chain (DC02) вимагає принципово іншого підходу до зберігання чутливої інформації.
4. Front-running (DC10) є специфічною загрозою блокчейн-середовища, без аналогу у веб-безпеці, і потребує архітектурних рішень (commit-reveal, приватні мемпули).
5. Незмінність смарт-контрактів підвищує вагу DC04 (Insecure Design) до рівня, вищого ніж в будь-якій традиційній системі.

Сформований перелік вимог DC01–DC10 може слугувати основою для проведення аудиту безпеки смарт-контрактів і процесу розробки децентралізованих додатків відповідно до принципів *Security by Design*.